Non-verbal Vocalization Control for Vietnamese Text-to-Speech

To Duy An, Nguyen Tung Luong[1]*, Tran Hong Nhat*[1]*, Do Van Hai*[1]
[1]*Viettel AI*
luongnt29@viettel.com.vn, nhatth2@viettel.com.vn, haidv21@viettel.com.vn

Abstract

Text-to-speech (TTS) systems can read Vietnamese aloud, but they cannot yet reliably make non-verbal sounds, such as laughing, sighing or coughing, at the point where the text asks for them. These sounds make synthetic speech feel human in voice assistants, customer-facing AI agents, audiobooks and videos. Two things stand in the way: existing systems ignore or garble such tags in Vietnamese, and no Vietnamese training data with labelled sounds exists. We build that data automatically by adapting the SynParaSpeech method to Vietnamese: recordings of real sounds are converted into each speaker's voice and inserted into natural pauses in real Vietnamese speech, with a matching tag in the transcript. The result is about 811 hours of labelled speech covering 17 sound types. Fine-tuning the open-source OmniVoice model on this data teaches it 15 new tags. The fine-tuned model makes the right sound at the right place for 50% of tags, against 11.5% for the released model, and misses only 35% of them, against 64%. Its word error rate on the tagged sentences falls from 8.1% to 6.7%; on plain text it rises a little, from 3.9% to 4.8%, with the same speaker similarity, and it runs at a real-time factor of 0.06, fast enough for live conversation.

Index Terms: text-to-speech, non-verbal vocalization, paralinguistics, Vietnamese, data synthesis

1. Introduction

When people talk, they communicate with more than words. A short laugh, a sigh, a breath before a long sentence or a hesitant "ừm…" tells the listener how the speaker feels and what they mean. These paralinguistic, or non-verbal, cues, which we call non-verbal vocalizations (NVVs), run through everyday conversation, and speech without them sounds flat and mechanical even when every word is pronounced correctly.

Text-to-speech (TTS) systems turn written text into spoken audio. Modern TTS models read text clearly in many voices, and some now let the user place a tag in the text, as in "Thật à [laughter] mình không ngờ luôn" ("Really? [laughter] I didn't expect that at all"), so that the voice laughs at exactly that point. A voice agent that answers customers can then chuckle at a joke or sigh in sympathy at a complaint, and that small reaction is often what makes a caller feel heard rather than processed; an audiobook or a dubbed video can let its narrator gasp or laugh where the script calls for it.

For Vietnamese, this control does not exist yet. What is needed is a TTS system that, given a Vietnamese sentence with a tag such as [cough], makes that sound at that point in the sentence, in the chosen voice, as naturally as a person would. To our knowledge, no available system can do this. Fish Audio's open model, OpenAudio S1 [1], offers markers such as (laughing) and (sighing) in 13 languages, none of them Vietnamese. OmniVoice [2] does speak Vietnamese, but its tags were learned from Chinese and English data: in our tests on Vietnamese sentences, it made no sound at all for 64% of the tags, and the right sound in the right place for only 11.5%. The natural remedy, training a model on Vietnamese speech in which each sound is labelled at its position, is blocked by a lack of data: to our knowledge, no public Vietnamese dataset of this kind exists.

This work closes both gaps. We start from OmniVoice, which already speaks Vietnamese and runs many times faster than real time, and teach it 15 new tags. Because no suitable data existed, we build it by adapting SynParaSpeech [3], a published method that creates such data automatically by inserting recorded sounds into real speech, to Vietnamese, and we extend it in five ways (Section 3.1). Our contributions are as follows:

– A Vietnamese NVV data pipeline that adapts and extends SynParaSpeech (Section 3.1).
– A Vietnamese NVV dataset of about 811 hours (224,064 utterances) covering 17 sound types.
– A fine-tuned OmniVoice that makes the right sound at the right place for 50% of tags, against 11.5% for the released model, at a small cost on plain text and with a real-time factor of 0.06.

2. Related Work

2.1. Zero-shot TTS

Zero-shot TTS clones a voice it was never trained on from a few seconds of reference audio. Neural codec language models such as VALL-E [4] generate discrete acoustic tokens one step at a time, while non-autoregressive models such as Voicebox [5], F5-TTS [6] and MaskGCT [7] generate speech by flow matching or masked prediction. CosyVoice [8] pairs a language model with a flow-matching decoder, and XTTS [9] extends zero-shot cloning to many languages. OmniVoice [2], the model we build on, scales this to over 600 languages with a non-autoregressive diffusion language model. For Vietnamese, viXTTS fine-tunes XTTS on the viVoice corpus [10], and other open systems fine-tune F5-TTS and similar models; we compare with four of them in Section 5. These systems read Vietnamese well, but none of them lets the user ask for a non-verbal sound in Vietnamese.

2.2. Non-verbal vocalization synthesis

A growing set of TTS systems accepts NVV tags in the text. Bark [11] responds to markers such as [laughs] and [sighs], CosyVoice [8] to tags such as [laughter] and [breath], and Fish Audio's OpenAudio S1 [1] to markers such as (laughing) and (sighing); OmniVoice has 13 inline tags, two of them vocalizations. Their tagged training data are mostly English and Chinese. Building such data is the main obstacle, and recent work automates it in two ways. One is to find and label NVVs in existing speech: NVSpeech [12] annotates Mandarin speech with 18 paralinguistic categories, and NonVerbalSpeech-38K [13] collects about 38k English and Chinese clips with 10 NVV types from films, shows and radio dramas. The other is to synthesize the data: SynParaSpeech [3] inserts voice-converted paralinguistic clips into natural conversational speech, giving 118.75 hours of Chinese data in 6 categories. We follow the synthesis route, because there is no Vietnamese corpus with NVVs to label, and extend SynParaSpeech for Vietnamese (Section 3.1).

3. Method

We synthesize NVV training data from real Vietnamese recordings (Section 3.1) and fine-tune OmniVoice on it (Section 3.2); Figure 1 gives an overview.

3.1. Training-data synthesis

Each training example is a real Vietnamese utterance with one NVV spliced into one of its pauses and a tag inserted at the same point in its transcript. It is built from a pool of NVV clips, collected once, in six steps per utterance.

– **NVV clips.** VocalSound [14] alone covers only six classes, and its crowdsourced clips often carry long silences or stray speech. We therefore pool clips from other NVV corpora and from events cut out of a tagged speech corpus, 17 classes in all against SynParaSpeech's 6. Every event is kept only if a sound event detector confirms its class, no speech overlaps it, its length is plausible and it is neither noisy nor clipped.
– **Pause detection.** Following SynParaSpeech, we first find the pauses where an NVV could go and only then let

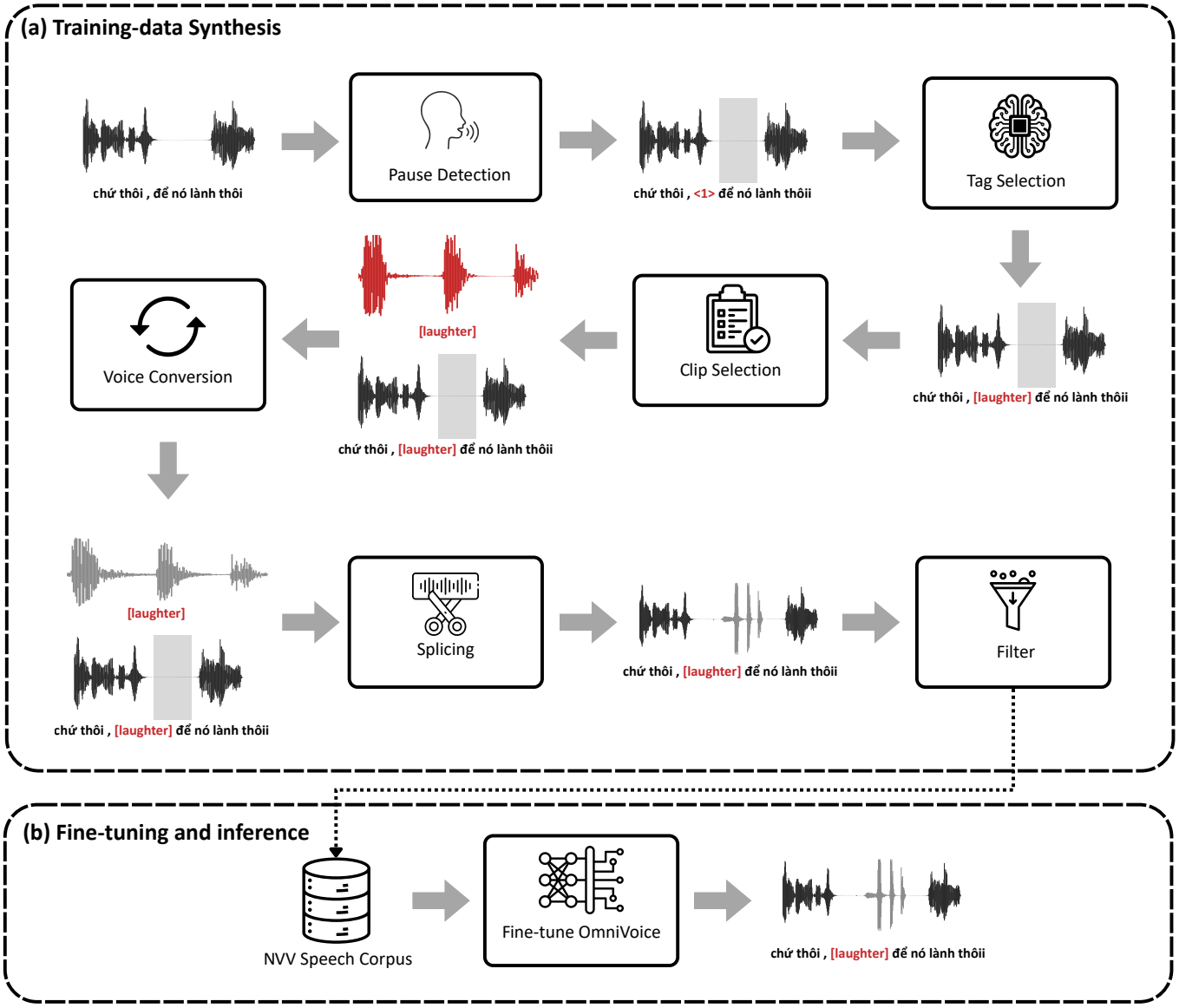


Figure 1: Overview. (a) Training-data synthesis on a real example: pause detection finds the pauses between words and marks them in the transcript; tag selection picks one of them and a tag, here [laughter]; clip selection draws a laughter clip that suits the speaker; voice conversion turns it into the speaker's voice; splicing inserts it into the pause; and the filter keeps the result only if its quality holds. (b) The resulting NVV speech corpus is used to fine-tune OmniVoice.

a model choose among them, since placing the tag first fails on connected speech. A voice activity detector finds the pauses, a forced aligner the word boundaries, and a pause between two words becomes a numbered slot in the transcript, e.g. "chứ thôi , <1> để nó lành thôi".

– **Tag selection.** An omni-modal LLM listens to the recording, reads the marked transcript, and picks the slot where a sound would most plausibly occur and the tag that fits, where SynParaSpeech uses a text-only model. The tag is inserted in code, so text and audio refer to the same point, and a tag balancer withholds any tag that holds too large a share.

– **Clip selection.** A clip is drawn at random, weighted towards clips that match the speaker on what voice conversion leaves unchanged: intensity, recording clarity, tempo (laughter only) and energy tilt, where SynParaSpeech picks one at random. With each distance d_a normalised and weighted by λ_a, a clip is drawn with probability proportional to $w = \exp(-\sum_a \lambda_a d_a)$, so a soft-spoken speaker still sometimes gets a loud laugh.

– **Voice conversion.** Seed-VC converts the clip into the speaker's voice, using the utterance itself as the reference, with pitch con-

Table 1: Synthetic NVV training set per tag, after quality filtering.

Tag	Utterances	Hours	Tag	Utterances	Hours
laughter	13,450	55.20	grunt	13,442	44.11
sigh	13,455	52.46	burp	13,435	41.82
cough	13,440	49.20	yawn	13,433	50.59
throatclearing	13,441	48.69	crying	13,433	51.01
sneeze	13,435	49.55	pant	13,432	49.94
sniff	13,522	51.70	yell	13,400	45.33
breathing	13,452	47.69	screaming	12,212	49.42
gasp	13,449	45.10	snoring	10,190	31.69
moan	13,443	47.33			
Total				224,064	810.82

ditioning off.

- **Splicing.** The event is set to the loudness of the speech before the cut, with a per-class offset, and inserted with short exponential fades. Each side gets the target silence $T = \bar{T}_p \, s_g \, s^{(c)}$, where $\bar{T}_p$ is the speaker's median pause and s_g and $s^{(c)}$ are global and per-class scales; silence is only added, filled with the recording's own background noise. SynParaSpeech does not describe this step.
- **Filter.** NISQA [15] scores both the source and the spliced recording, and an example is kept only if neither its quality nor its continuity drops past a threshold, where SynParaSpeech relies on manual review.

3.2. Fine-tuning and inference

OmniVoice [2] is a massively multilingual zero-shot TTS model that maps text to multi-codebook acoustic tokens with a non-autoregressive diffusion language model and clones a voice from a reference recording. As released, it cannot generate Vietnamese NVVs, for three reasons. Its tag set is small: only two of its inline tags, [laughter] and [sigh], are non-verbal sounds, so 15 of our 17 classes have no tag. It has never seen a tag in Vietnamese: its tagged data come from Chinese and English corpora. And it was not trained for tag control, which its paper does not evaluate. We therefore fine-tune it on our synthetic data alone, which supplies all 17 tags on Vietnamese speech. No untagged speech is mixed in: each utterance is long and carries a single tag, so most of it is ordinary speech, and the same data teaches both the tags and plain synthesis. The training objective is OmniVoice's own, and at inference the tag is written inline in the input text.

4. Experimental Setup

4.1. Data

Training uses only the synthetic NVV set, built from ViVoice [10]: 224,064 utterances (810.8 h), each with one event, with 10,190 to 13,522 utterances per tag (Table 1).

4.2. Training

We initialise from the public OmniVoice checkpoint and fine-tune all parameters with the default settings of OmniVoice's training configuration for 10,000 steps, on NVIDIA B200 GPUs.

4.3. Baselines

We evaluate two tasks. In zero-shot TTS, the input is plain Vietnamese text without tags; this task checks that fine-tuning keeps ordinary synthesis intact. Its baselines are the released OmniVoice, four Vietnamese TTS systems (viXTTS, F5-TTS, VietVoice-TTS and VieNeu-TTS) and Fish Audio's TTS, queried through an API served with vLLM. In NVV synthesis, the input is tagged text, and each tag must become the matching vocalization at its position; its baselines are the released OmniVoice and Fish Audio. The listening test covers Fish Audio and both OmniVoice models.

4.4. Evaluation

For zero-shot TTS, every system is evaluated on the same 238 utterances sampled

Table 2: Zero-shot Vietnamese TTS on the same 238 plain-text utterances. WER in %. Fish Audio is queried through an API served with vLLM, not run as a local model, so its RTF is not measured.

System	**WER ↓**	**SIM-o ↑**	**RTF ↓**
viXTTS	3.27	0.70	0.67
F5-TTS	3.02	0.82	0.47
VietVoice-TTS	2.37	0.78	0.47
VieNeu-TTS	2.00	0.65	0.89
Fish Audio	11.31	0.79	–
OmniVoice			
+ base	3.93	0.84	0.06
+ fine-tuned (ours)	4.78	0.84	0.06

Table 3: Vietnamese NVV synthesis over 2,940 tagged items (14 classes). Fish Audio is evaluated on a 280-item subset, through an API served with vLLM, so its RTF is not measured. WER in %.

System	**NVMR ↓**	**NVPA ↑**	**WER ↓**	**SIM-o ↑**	**RTF ↓**
Fish Audio	0.6543	0.2280	60.48	0.32	–
OmniVoice					
+ base	0.6435	0.1153	8.11	0.75	0.06
+ fine-tuned (ours)	0.3507	0.5000	6.73	0.74	0.06

Table 4: Listening test on NVV synthesis (1–5, with 95% confidence intervals).

System	**MOS-Q ↑**	**MOS-N ↑**	**MOS-E ↑**
Fish Audio	3.38 ± 0.04	2.12 ± 0.05	1.96 ± 0.04
OmniVoice			
+ base	3.63 ± 0.03	2.19 ± 0.04	1.56 ± 0.03
+ fine-tuned (ours)	3.89 ± 0.01	3.15 ± 0.04	2.96 ± 0.05

from VietVoice, an internal Vietnamese speech dataset. For NVV synthesis, we use 420 real Vietnamese sentences, 30 for each of 14 classes, each chosen so that the tag fits its content; each is synthesised with 7 reference voices that are not in the training data, giving 2,940 items per system. For both OmniVoice models we tokenise the whole tagged string at inference, as in training.

NV placement accuracy (NVPA, 0–1, higher is better) is the share of tags for which a sound event detector finds a vocalization of the tagged class at the tag's position. The NV miss rate (NVMR, 0–1, lower is better) is the share of tags with no vocalization at all. Word error rate (WER, in %, lower is better) compares a Vietnamese ASR transcript of the output with the input text. SIM-o (0–1, higher is better) is the cosine between the speaker embeddings of the output and of the reference voice. The real-time factor (RTF, lower is better) is the time to synthesise an utterance divided by its duration, on one NVIDIA A100 GPU. In the listening test, ten native Vietnamese listeners each rate 40 NVV samples, balanced across the three systems, from 1 (bad) to 5 (excellent) on quality (MOS-Q), naturalness (MOS-N) and expressiveness (MOS-E), the last asking whether the vocalization is the one the tag asks for and carries the feeling of the sentence.

5. Results

Given Vietnamese text, the released OmniVoice produces no vocalization at 37% of the [laughter] and [sigh] tags, the two it supports, and at 69% of the other 12; even its own two tags behave very differently, with a correct laugh for 79% of [laughter] tags but a correct sigh for only 1% of [sigh] tags. Fine-tuning on our data changes this (Table 3). The share of missed tags falls from 64% to 35%, and the share realised correctly rises from 11.5% to 50%: from 40% to 86% on laughter and sigh and from 7% to 44% on the new tags. The

word error rate on the tagged sentences also falls, from 8.11% to 6.73%, and speaker similarity stays close (0.74 against 0.75). On plain text (Table 2), the cost is small: the word error rate rises from 3.93% to 4.78%, speaker similarity is 0.84 for both models, and the RTF stays at 0.06. Fish Audio does worse than our model on both tasks: a word error rate of 11.31% on plain text and, on NVV synthesis, 65% of tags missed and 23% correct. Its high word error rate (60.48%) and low speaker similarity (0.32) on NVV synthesis share one cause: after a tag, whether or not it produced the sound, Fish Audio often drops the rest of the sentence and leaves a long pause in its place. Listeners prefer the fine-tuned model on all three scores (Table 4). Its naturalness rises from 2.19 to 3.15 and its expressiveness from 1.56 to 2.96, against 2.12 and 1.96 for Fish Audio, and its audio quality is the highest of the three (3.89, against 3.63 and 3.38).

6. Conclusion

We set out to give Vietnamese TTS the ability to laugh, sigh, cough and make other non-verbal sounds where the text asks for them. We built a pipeline that creates the missing training data automatically from real Vietnamese speech, producing about 811 hours over 17 sound types. Fine-tuned on this data, OmniVoice makes the right sound at the right place for 50% of tags instead of 11.5%, misses 35% instead of 64%, and reads tagged sentences more accurately, at a small cost on plain text and with a real-time factor of 0.06; listeners rate its naturalness and expressiveness well above both the released model and Fish Audio.

The work has limits. Each training example carries one sound, inserted into an existing pause, so sounds that overlap speech, such as laughing while talking, are not covered. The inserted sounds are recorded by other people and only converted into the target voice. Burp, grunt and yell are trained but not yet tested. Next, we plan to train a version on ViVoice alone to separate the effect of the NVV data from that of extra Vietnamese training, and to extend the test set to all 17 sound types.

7. References

[1] Fish Audio. OpenAudio S1. `https://huggingface.co/fishaudio/openaudio-s1-mini`, 2025.

[2] Han Zhu, Lingxuan Ye, Wei Kang, Zengwei Yao, Liyong Guo, Fangjun Kuang, Zhifeng Han, Weiji Zhuang, Long Lin, and Daniel Povey. OmniVoice: Towards omnilingual zero-shot text-to-speech with diffusion language models. *arXiv preprint arXiv:2604.00688*, 2026.

[3] Bingsong Bai, Qihang Lu, et al. SynParaSpeech: Automated synthesis of paralinguistic datasets for speech generation and understanding. *arXiv preprint arXiv:2509.14946*, 2025.

[4] Chengyi Wang, Sanyuan Chen, Yu Wu, Ziqiang Zhang, Long Zhou, Shujie Liu, Zhuo Chen, Yanqing Liu, Huaming Wang, Jinyu Li, Lei He, Sheng Zhao, and Furu Wei. Neural codec language models are zero-shot text to speech synthesizers. *arXiv preprint arXiv:2301.02111*, 2023.

[5] Matthew Le et al. Voicebox: Text-guided multilingual universal speech generation at scale. In *Proc. NeurIPS*, 2023.

[6] Yushen Chen, Zhikang Niu, Ziyang Ma, Keqi Deng, Chunhui Wang, Jian Zhao, Kai Yu, and Xie Chen. F5-TTS: A fairytaler that fakes fluent and faithful speech with flow matching. *arXiv preprint arXiv:2410.06885*, 2024.

[7] Yuancheng Wang et al. MaskGCT: Zero-shot text-to-speech with masked generative codec transformer. *arXiv preprint arXiv:2409.00750*, 2024.

[8] Zhihao Du et al. CosyVoice: A scalable multilingual zero-shot text-to-speech synthesizer based on supervised semantic tokens. *arXiv preprint arXiv:2407.05407*, 2024.

[9] Edresson Casanova et al. XTTS: a massively multilingual zero-shot text-to-speech model. In *Proc. Interspeech*, 2024.

[10] capleaf. viVoice: Enabling Vietnamese multi-speaker speech synthesis. `https://huggingface.co/datasets/capleaf/viVoice`, 2024.

[11] Suno. Bark. `https://github.com/suno-ai/bark`, 2023.

[12] Huan Liao, Qinke Ni, Yuancheng Wang, Yiheng Lu, Haoyue Zhan, Pengyuan Xie, Qiang Zhang, and Zhizheng Wu. NVSpeech: An integrated and scalable pipeline for human-like speech modeling with paralinguistic vocalizations. *arXiv preprint arXiv:2508.04195*, 2025.

[13] Runchuan Ye, Yixuan Zhou, Renjie Yu, Zijian Lin, Kehan Li, Xiang Li, Xin Liu, Guoyang Zeng, and Zhiyong Wu. A scalable pipeline for enabling non-verbal speech generation and understanding. *arXiv preprint arXiv:2508.05385*, 2025.

[14] Yuan Gong, Jin Yu, and James Glass. VocalSound: A dataset for improving human vocal sounds recognition. In *Proc. ICASSP*, 2022.

[15] Gabriel Mittag, Babak Naderi, Assmaa Chehadi, and Sebastian Möller. NISQA: A deep CNN-self-attention model for multidimensional speech quality prediction with crowdsourced datasets. In *Proc. Interspeech*, 2021.